



Faculty of Engineering
Introduction to Machine Learning
Laboratory Manual

NAME:

ENROLLMENT NUMBER:

SUBJECT NAME:

SUBJECT CODE: BRANCH:

SEMESTER: ACADEMIC YEAR:



Swaminarayan university
Publication

ISBN Number: 978-81-998196-4-1



सर्व विद्या अधिपतये

Certificate

This is to certify that Mr. / Ms. / Mrs.
having enrollment number of Semester ME /
BE / DIPLOMA in.....has satisfactorily completed his
term working (Subject name) (subject code:.....)
during academic year at Faculty of Engineering, Swaminarayan University.

Place:

Date:

ASHWINI WALAKE
SUBJECT IN-CHARGE

PROF. RADHIKA GOKANI
HOD

DR. BHAVNA SONI
DEAN

List of Practical

Sr. No.	Title	Page No.	Date	Sign
1	Explore any one machine learning tool.(like Weka, Tensorflow, Scikit-learn, Colab, etc.)			
2	Write a NumPy program to implement following operation - to convert a list of numeric values into a one-dimensional NumPy array - to create a 3x3 matrix with values ranging from 2 to 10 - to append values at the end of an array - to create another shape from an array without changing its data(3*2 to 2*3)			
3	Write a NumPy program to implement following operation -to split an array of 14 elements into 3 arrays, each with 2, 4, and 8 elements in the original order -to stack arrays horizontally (column wise).			
4	Write a NumPy program to implement following operation -to add, subtract, multiply, divide arguments element-wise -to round elements of the array to the nearest integer - to calculate mean across dimension, in a 2D numpy array - to calculate the difference between neighboring elements, elementwise of a given array.			
5	Write a NumPy program to implement following operation -to find the maximum and minimum value of a given flattened array - to compute the mean, standard deviation, and variance of a given array along the second axis.			
6	Write a Pandas program to implement following operation -to convert a NumPy array to a Pandas series - to convert the first column of a Data Frame as a Series - to create the mean and standard deviation of the data of a given Series -to sort a given Series.			
7	Write a Pandas program to implement following operation -to create a dataframe from a dictionary and display it - to sort the DataFrame first by 'name' in ascending order - to delete the one			

	specific column from the DataFrame - to write a DataFrame to CSV file using tab separator			
8	Write a Pandas program to create a line plot of the opening, closing stock prices of given company between two specific dates.			
9	Write a Pandas program to create a plot of Open, High, Low, Close, Adjusted Closing prices and Volume of given company between two specific dates			
10	Write a Pandas program to implement following operation - to find and drop the missing values from the given dataset - to remove the duplicates from the given dataset.			
11	Write a Pandas program to filter all columns where all entries present, check which rows and columns has a NaN and finally drop rows with any NaNs from the given dataset.			
12	Write a Python program using Scikit-learn to print the keys, number of rows-columns, feature names and the description of the given data.			
13	Write a Python program to implement K-Nearest Neighbour supervised machine learning algorithm for given dataset.			
14	Write a Python program to implement a machine learning algorithm for given dataset. (It is recommended to assign different machine learning algorithms group wise – micro project)			

Practical-1

AIM: Explore any one machine learning tool. (Like Weka, Tensor flow, Scikit-learn, Colab, etc.)

Introduction

Scikit-learn is an open-source Python library for machine learning. It features various classification, regression, and clustering algorithms, including support vector machines, random forests, gradient boosting, k-means, and DBSCAN. It is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.

Here are some of the main features of Scikit-learn:

- **Simple and efficient tools for data mining and data analysis.** It provides a selection of efficient tools for machine learning and statistical modeling including classification, regression, clustering and dimensionality reduction via a consistent interface in Python.
- **Accessible to everybody, and reusable in various contexts.** Built on NumPy, SciPy, and matplotlib.
- **Built on NumPy, SciPy, and Matplotlib.** Scikit-learn is not only open source but also commercially usable - BSD license.

Scikit-learn comes with a few standard datasets, for instance, the iris and digits datasets for classification and the Boston house prices dataset for regression. Scikit-learn also features various model selection and evaluation tools, including cross-validation.

In addition to its powerful features for machine learning, Scikit-learn also includes a lot of utility functions for common tasks in data science such as preprocessing data, fine-tuning model parameters, and evaluating models.

Overall, Scikit-learn is a versatile library that provides a lot of functionality out of the box.

Practical-2

AIM : Write a NumPy program to implement following operation

- **to convert a list of numeric values into a one-dimensional NumPy array**

CODE:

```
import numpy as np
list_of_numbers = [1, 2, 3, 4, 5]

numpy_array = np.array(list_of_numbers)

print(numpy_array)
```

OUTPUT:

```
array([1, 2, 3, 4, 5])
```

- **to create a 3x3 matrix with values ranging from 2 to 10**

CODE:

```
import numpy as np

matrix = np.arange(2, 11).reshape(3,3)

print(matrix)
```

OUTPUT:

```
[[ 2  3  4]
 [ 5  6  7]
 [ 8  9 10]]
```

- **to append values at the end of an array**

CODE:

```
import numpy as np
arr = np.array([1, 2, 3])
values = np.array([4, 5, 6])
arr = np.append(arr, values)
print(arr)
```

OUTPUT:

```
[1 2 3 4 5 6]
```

- **to create another shape from an array without changing its data(3*2 to 2*3)**
-

CODE:

```
import numpy as np
arr = np.array([[1, 2], [3, 4], [5, 6]])
print("Original array:")
print(arr)

reshaped_arr = arr.reshape(2, 3)
print("\nReshaped array:")
print(reshaped_arr)
```

OUTPUT

```
Original array:
[[1 2]
 [3 4]
 [5 6]]
```

```
Reshaped array:
[[1 2 3]
 [4 5 6]]
```

Practical-3

AIM: Write a NumPy program to implement following operation

- **to split an array of 14 elements into 3 arrays, each with 2, 4, and 8 elements in the original order**

CODE:

```
import numpy as np
x = np.arange(1, 15)
print("Original array:",x)
print("After splitting:")
print(np.split(x, [2, 6]))
```

OUTPUT:

```
Original array: [ 1  2  3  4  5  6  7  8  9 10 11 12 13 14]
After splitting:[array([1, 2]), array([3, 4, 5, 6]),
array([ 7,  8,  9, 10, 11, 12, 13, 14])]
```

- **to stack arrays horizontally (column wise)**

CODE:

```
import numpy as np

ar1 = np.array([1, 2, 3])
ar2 = np.array([4, 5, 6])

ar_h = np.hstack((ar1, ar2))

print(ar_h)
```

OUTPUT:

```
[1 2 3 4 5 6]
```

Practical-4

AIM : Write a NumPy program to implement following operation

- **to add, subtract, multiply, divide arguments element-wise**

CODE: Multiply

```
import numpy as np
A = [[1, 2, 3],
      [4, 5, 6]]
B = [[1, 5],
      [2, 3],
      [4, 6]]

result = np.dot(A, B)

for r in result:
    print(r)
```

OUTPUT:

```
[17 29]
[38 71]
```

CODE: Addition, Subtraction, Division

```
import numpy as np

# Assuming 'matrix1' and 'matrix2' are your numpy arrays
matrix1 = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
matrix2 = np.array([[10, 11, 12], [13, 14, 15], [16, 17, 18]])

# Add the matrices
result = np.add(matrix1, matrix2)
result1 = np.subtract(matrix1, matrix2)
result2 = np.divide(matrix1, matrix2)

print("Addition : \n", result, )
print()
print("Subtraction : \n", result1, )
print()
print("Division : \n", result2 )
```

OUTPUT:

Addition :

```
[[11 13 15]
 [17 19 21]
 [23 25 27]]
```

Subtraction :

```
[[ -9 -9 -9]
 [ -9 -9 -9]
 [ -9 -9 -9]]
```

Division :

```
[[0.1      0.18181818 0.25    ]
 [0.30769231 0.35714286 0.4     ]
 [0.4375    0.47058824 0.5     ]]
```

- **to round elements of the array to the nearest integer**
CODE:

```
import numpy as n

# create array
y = n.array([0.2, 0.3, 0.4, 0.5, 0.6, 0.7])
print("Original array:", end=" ")
print(y)

# round to nearest integer
y = n rint(y)
print("After rounding off:", end=" ")
print(y)
```

OUTPUT:

```
Original array: [0.2 0.3 0.4 0.5 0.6 0.7]
After rounding off: [0. 0. 0. 0. 1. 1.]
```

- **to calculate mean across dimension, in a 2D numpy array**
CODE:

```
import numpy as np

# creating 2d array
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Calculating mean across Rows
row_mean = np.mean(arr, axis=1)

row1_mean = row_mean[0]
print("Mean of Row 1 is", row1_mean)

row2_mean = row_mean[1]
```

```
print("Mean of Row 2 is", row2_mean)

row3_mean = row_mean[2]
print("Mean of Row 3 is", row3_mean)

# Calculating mean across Columns
column_mean = np.mean(arr, axis=0)

column1_mean = column_mean[0]
print("Mean of column 1 is", column1_mean)

column2_mean = column_mean[1]
print("Mean of column 2 is", column2_mean)

column3_mean = column_mean[2]
print("Mean of column 3 is", column3_mean)
```

OUTPUT:

```
Mean of Row 1 is 2.0
Mean of Row 2 is 5.0
Mean of Row 3 is 8.0
Mean of column 1 is 4.0
Mean of column 2 is 5.0
Mean of column 3 is 6.0
```

- **to calculate the difference between neighboring elements, element-wise of a given array**

CODE:

```
import numpy as np

# create a numpy 1d-array
arr = np.array([1, 12, 3, 14, 5,
16, 7, 18, 9, 110])

# finding the difference between
# neighboring elements
result = np.diff(arr)

print(result)
```

OUTPUT:

```
[ 11 -9 11 -9 11 -9 11 -9 101]
```

Practical-5

AIM : Write a NumPy program to implement following operation

- **to find the maximum and minimum value of a given flattened array**

CODE:

```
import numpy as np

# Assuming 'arr' is your numpy array
arr = np.array([[1, 2], [3, 4]])

# Flatten the array
flat_arr = arr.flatten()

# Find the maximum value
max_val = np.amax(flat_arr)

# Find the minimum value
min_val = np.amin(flat_arr)

print("Maximum Value:", max_val)
print("Minimum Value:", min_val)
```

OUTPUT:

```
Maximum Value: 4
Minimum Value: 1
```

- **to compute the mean, standard deviation, and variance of a given array along the second axis**

CODE:

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
mean = np.mean(arr, axis=1)
print("Mean: ", mean)
std_dev = np.std(arr, axis=1)
print("Standard Deviation: ", std_dev)
variance = np.var(arr, axis=1)
print("Variance: ", variance)
```

OUTPUT:

```
Mean: [2. 5. 8.]
Standard Deviation: [0.81649658 0.81649658 0.81649658]
Variance: [0.66666667 0.66666667 0.66666667]
```

Practical-6

AIM : Write a Pandas program to implement following operation

- **to convert a NumPy array to a Pandas series**

CODE:

```
import pandas as pd
data = [1,2,3,4,5]
df = pd.DataFrame(data)
print(df)
```

OUTPUT:

```
0
0 1
1 2
2 3
3 4
4 5
```

- **to convert the first column of a DataFrame as a Series**

CODE:

```
import pandas as pd

# Assuming df is your DataFrame
df = pd.DataFrame({
    'A': ['foo', 'bar', 'baz'],
    'B': ['one', 'one', 'two'],
    'C': ['x', 'y', 'z'],
    'D': [1, 2, 3]
})

# Use iloc to select the first column and convert it to a Series
series = df.iloc[:, 2]

print(series)
```

OUTPUT:

```
0  x
1  y
2  z
Name: C, dtype: object
```

- **to create the mean and standard deviation of the data of a given Series**

CODE:

```
import pandas as pd

# Assuming 's' is your pandas Series
s = pd.Series([1, 2, 3, 4, 5])
```

```
mean = s.mean()
std_dev = s.std()

print(f"Mean: {mean}, Standard Deviation: {std_dev}")
```

OUTPUT:

Mean: 3.0, Standard Deviation: 1.5811388300841898

- **to sort a given Series**

CODE:

```
import pandas as pd

# Create a series
s = pd.Series([3, 1, 2, 5, 4], index=['c', 'a', 'b', 'e', 'd'])

# Sort the series
sorted_s = s.sort_values()

print(sorted_s)
```

OUTPUT:

```
a    1
b    2
c    3
d    4
e    5
dtype: int64
```

Practical-7

AIM : Write a Pandas program to implement following operation

- **to create a dataframe from a dictionary and display it**

CODE:

```
import pandas as pd

data = {
    'Name': ['John', 'Anna', 'Peter'],
    'Age': [28, 24, 22],
    'City': ['New York', 'London', 'Paris']
}

df = pd.DataFrame(data)
print(df)
```

OUTPUT:

	Name	Age	City
0	John	28	New York
1	Anna	24	London
2	Peter	22	Paris

- **to sort the DataFrame first by 'name' in ascending order**

CODE:

```
import pandas as pd

age_list = [['Afghanistan', 1952, 8425333, 'Asia'],
            ['Australia', 1957, 9712569, 'Oceania'],
            ['Brazil', 1962, 76039390, 'Americas'],
            ['China', 1957, 637408000, 'Asia'],
            ['France', 1957, 44310863, 'Europe'],
            ['India', 1952, 3.72e+08, 'Asia'],
            ['United States', 1957, 171984000, 'Americas']]

df = pd.DataFrame(age_list, columns=['Country_Name', 'Year',
    'Population', 'Continent'])
df.sort_values(by=['Country_Name'])
df
```

OUTPUT:

	Country_Name	Year	Population	Continent
0	Afghanistan	1952	8425333.0	Asia

	Country_Name	Year	Population	Continent
1	Australia	1957	9712569.0	Oceania
2	Brazil	1962	76039390.0	Americas
3	China	1957	637408000.0	Asia
4	France	1957	44310863.0	Europe
5	India	1952	372000000.0	Asia
6	United States	1957	171984000.0	Americas

- to delete the one specific column from the DataFrame

CODE:

```
# Import pandas package
import pandas as pd

# create a dictionary with five fields each
data = {
'A': ['A1', 'A2', 'A3', 'A4', 'A5'],
'B': ['B1', 'B2', 'B3', 'B4', 'B5'],
'C': ['C1', 'C2', 'C3', 'C4', 'C5'],
'D': ['D1', 'D2', 'D3', 'D4', 'D5'],
'E': ['E1', 'E2', 'E3', 'E4', 'E5']}

# Convert the dictionary into DataFrame
df = pd.DataFrame(data)

df
```

OUTPUT:

	A	B	C	D	E
0	A1	B1	C1	D1	E1
1	A2	B2	C2	D2	E2
2	A3	B3	C3	D3	E3
3	A4	B4	C4	D4	E4
4	A5	B5	C5	D5	E5

Practical-8

AIM : Write a Pandas program to create a line plot of the opening, closing stock prices of given company between two specific dates

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the data
df = pd.read_csv('alphabet_stock_data.csv')

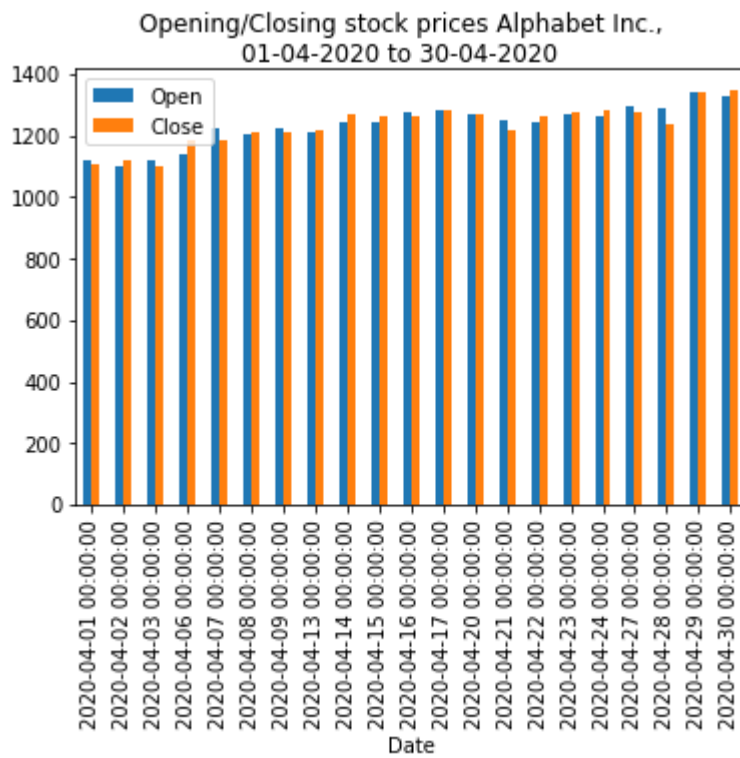
# Ensure that the 'Date' column is a datetime object
df['Date'] = pd.to_datetime(df['Date'])

# Set 'Date' as the index
df.set_index('Date', inplace=True)

# Filter data for a specific company and between two dates
filtered_df = df[(df['Company'] == 'Company_Name') &
                 (df['Date'] >= 'start_date') &
                 (df['Date'] <= 'end_date')]

# Create the line plot
plt.figure(figsize=(10,5))
plt.plot(filtered_df['Open'], label='Open')
plt.plot(filtered_df['Close'], label='Close')
plt.title('Opening and Closing Stock Prices')
plt.xlabel('Date')
plt.ylabel('Price')
plt.legend()
plt.show()
```

OUTPUT:



Practical-9

AIM : Write a Pandas program to create a plot of Open, High, Low, Close, Adjusted Closing prices and Volume of given company between two specific dates.

CODE:

```
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt

# Input company ticker and date range
company = "AAPL"      # Apple Inc.
start_date = "2023-01-01"
end_date = "2023-12-31"

# Download stock data
df = yf.download(company, start=start_date, end=end_date)

# Display first five records
print(df.head())

# Plot Open, High, Low, Close, Adjusted Close
plt.figure(figsize=(14, 8))

plt.plot(df.index, df['Open'], label='Open')
plt.plot(df.index, df['High'], label='High')
plt.plot(df.index, df['Low'], label='Low')
plt.plot(df.index, df['Close'], label='Close')

# Plot Adjusted Close (if available)
if 'Adj Close' in df.columns:
    plt.plot(df.index, df['Adj Close'], label='Adjusted Close')

plt.title(f'{company} Stock Prices')
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.grid(True)
plt.show()

# Plot Volume
plt.figure(figsize=(14, 4))
plt.plot(df.index, df['Volume'], color='purple')
plt.title(f'{company} Trading Volume')
plt.xlabel("Date")
plt.ylabel("Volume")
plt.grid(True)
plt.show()
```

OUTPUT:

	Open	High	Low	Close	Adj Close	Volume	
	Date						
	2023-01-03	130.28	130.90	124.17	125.07	123.33	112117500
	2023-01-04	126.89	128.66	125.08	126.36	124.60	89113600
	2023-01-05	127.13	127.77	124.76	125.02	123.28	80962700
	2023-01-06	126.01	130.29	124.89	129.62	127.82	87754700
2023-01-09	130.47	133.41	129.89	130.15	128.34	70790800	

Practical-10

AIM : Write a Pandas program to implement following operation

- **to find and drop the missing values from the given dataset**

CODE:

```
importing pandas as pd
import pandas as pd

# importing numpy as np
import numpy as np

# dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
        'Second Score': [30, 45, 56, np.nan],
        'Third Score':[np.nan, 40, 80, 98]}

# creating a dataframe from list
df = pd.DataFrame(dict)

# using isnull() function
df.isnull()
```

OUTPUT:

	First Score	Second Score	Third Score
0	False	False	True
1	False	False	False
2	True	False	False
3	False	True	False

- **to remove the duplicates from the given dataset**

CODE:

```
import pandas as pd

data = {
    "A": ["TeamA", "TeamB", "TeamB", "TeamC", "TeamA"],
    "B": [50, 40, 40, 30, 50],
    "C": [True, False, False, False, True]
}

df = pd.DataFrame(data)
```

```
display(df.drop_duplicates())
```

OUTPUT:

	A	B	C
0	TeamA	50	True
1	TeamB	40	False
3	TeamC	30	False

Practical-11

AIM: Write a Pandas program to filter all columns where all entries present, check which rows and columns has a NaN and finally drop rows with any NaNs from the given dataset

CODE:

```
import pandas as pd
import numpy as np

# Assuming df is your DataFrame
df = pd.DataFrame({
    'A': [1, 2, np.nan],
    'B': [5, np.nan, np.nan],
    'C': [1, 2, 3]
})

print("Original DataFrame:")
print(df)

# Filter all columns where all entries present
df_no_nan = df.dropna(axis=1)
print("\nDataFrame after dropping columns with NaN values:")
print(df_no_nan)

# Check which rows and columns has a NaN
print("\nRows where NaN values exist:")
print(df[df.isna().any(axis=1)])

print("\nColumns where NaN values exist:")
print(df.columns[df.isna().any()].tolist())

# Drop rows with any NaNs
df_dropped = df.dropna()
print("\nDataFrame after dropping rows with NaN values:")
print(df_dropped)
```

OUTPUT:

Original DataFrame:

```
A  B  C
0  1.0  5.0  1
1  2.0  NaN  2
2  NaN  NaN  3
```

DataFrame after dropping columns with NaN values:

```
C
0  1
1  2
```

2 3

Rows where NaN values exist:

```
A B C
1 2.0 NaN 2
2 NaN NaN 3
```

Columns where NaN values exist:

```
['A', 'B']
```

DataFrame after dropping rows with NaN values:

```
A B C
0 1.0 5.0 1
```

Practical-12

AIM : Write a Python program using Scikit-learn to print the keys, number of rows-columns, feature names and the description of the given data.

CODE:

```
from sklearn import datasets

# Load iris dataset
iris = datasets.load_iris()

# Print keys
print("Keys:", iris.keys())

# Print number of rows and columns
print("Shape:", iris.data.shape)

# Print feature names
print("Feature names:", iris.feature_names)

# Print description
print("Description:", iris.DESCR)
```

OUTPUT:

```
Keys: dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_m
odule'])
Shape: (150, 4)
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Description: .. _iris_dataset:
```

```
Iris plants dataset
-----
```

```
**Data Set Characteristics:**
```

```
:Number of Instances: 150 (50 in each of three classes)
```

```
:Number of Attributes: 4 numeric, predictive attributes and the class
```

```
:Attribute Information:
```

```
- sepal length in cm
```

```
- sepal width in cm
```

```
- petal length in cm
```

```
- petal width in cm
```

```
- class:
```

```
- Iris-Setosa
```

```
- Iris-Versicolour
```

```
- Iris-Virginica
```

Summary Statistics:

```
=====
```

Min Max Mean SD Class Correlation
=====

sepal length:	4.3	7.9	5.84	0.83	0.7826
sepal width:	2.0	4.4	3.05	0.43	-0.4194
petal length:	1.0	6.9	3.76	1.76	0.9490 (high!)
petal width:	0.1	2.5	1.20	0.76	0.9565 (high!)

=====

:Missing Attribute Values: None
:Class Distribution: 33.3% for each of 3 classes.
:Creator: R.A. Fisher
:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
:Date: July, 19

Practical-13

AIM: Write a Python program to implement K-Nearest Neighbour supervised machine learning algorithm for given dataset.

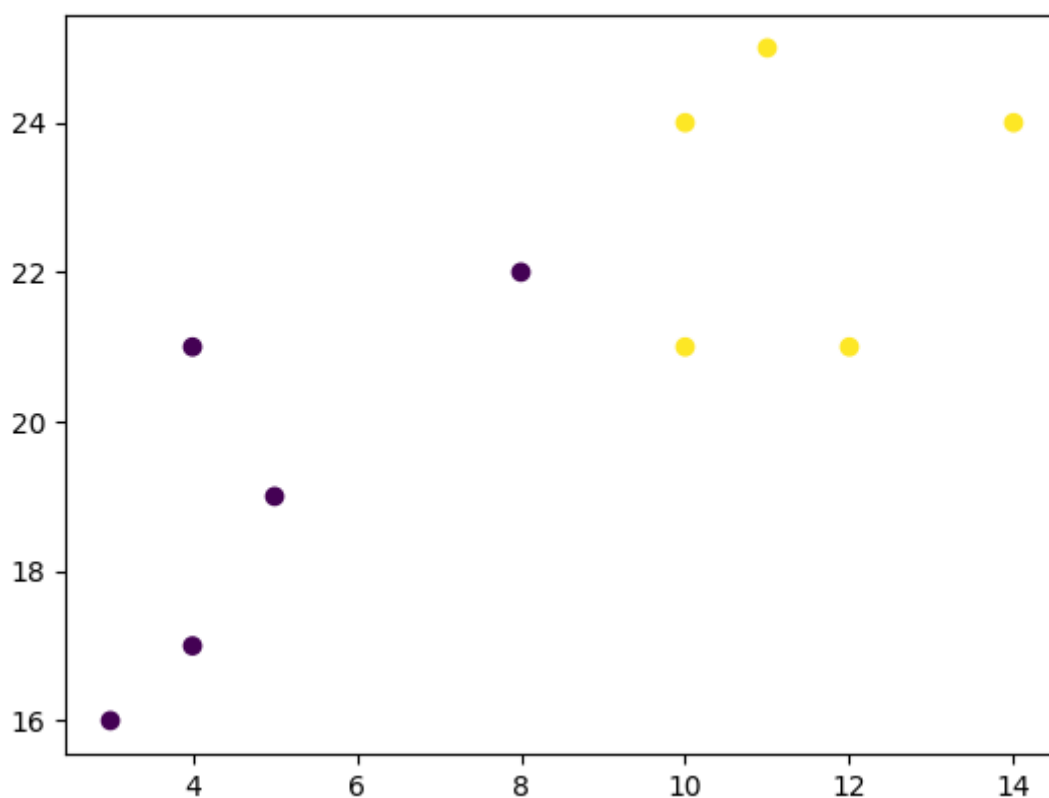
CODE:

```
import matplotlib.pyplot as plt

x = [4, 5, 10, 4, 3, 11, 14, 8, 10, 12]
y = [21, 19, 24, 17, 16, 25, 24, 22, 21, 21]
classes = [0, 0, 1, 0, 0, 1, 1, 0, 1, 1]

plt.scatter(x, y, c=classes)
plt.show()
```

OUTPUT:



Practical-14

AIM: Write a Python program to implement a machine learning algorithm for given dataset.

CODE:

DATASET USED: Iris dataset.

```
# Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load the Iris dataset
iris = load_iris()

# Features (X) and Target (y)
X = iris.data
y = iris.target

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train the model
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)

# Print predictions
print("Predicted Values:")
print(y_pred)

# Print actual values
print("\nActual Values:")
print(y_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

OUTPUT:

Predicted Values:

`[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]`

Actual Values:

`[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]`

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30